Cahier des charges

API REST de gestion d’épargne à objectif

LICELI Technologies

Stage technicien — DIPAMA Gamine — 2026

Sommaire

Sommaire 2

Glossaire et liste des sigles 3

1. Contexte 4

2. Problématique 4

3. Objectifs du projet 4

4. Description fonctionnelle 5

5. Périmètre 6

6. Contraintes 6

7. Livrables attendus 7

8. Critères d'évaluation 7

9. Point d'attention techniques identifiées en cours de projet 7

Table des matières 8

Glossaire et liste des sigles

Terme	Définition
Tirelire	Compte d'épargne dédié à un objectif précis, associé à un montant cible et une date d'échéance
Épargne à objectif	Produit financier permettant à un client d'épargner en vue d'un projet défini, avec blocage des retraits jusqu'à échéance ou objectif atteint
UEMOA	Union Économique et Monétaire Ouest Africaine. Zone économique regroupant plusieurs pays d'Afrique de l'Ouest, dont le Burkina Faso
API REST	Interface de programmation permettant à des applications de communiquer entre elles via le protocole HTTP, selon les principes de l'architecture REST
Cloisonnement des données	Principe garantissant qu'un client ne peut accéder qu'à ses propres données, jamais à celles d'un autre client
Migration (Django)	Fichier généré automatiquement décrivant les évolutions du schéma de la base de données, à appliquer pour synchroniser la base avec les modèles définis dans le code

1. Contexte

LICELI Technologies conçoit et édite des solutions logicielles pour les établissements financiers de la zone UEMOA, notamment dans les domaines de l'audit interne, du contrôle permanent, de la conformité, de la gestion des incidents et des services de mobile banking.

L'épargne à objectif est un produit d'inclusion financière répandu dans la sous-région. Son principe : un client ouvre un compte dédié à un projet précis (une "tirelire"), y effectue des versements libres, et ne peut retirer les fonds qu'une fois l'objectif atteint ou l'échéance arrivée. Derrière cette simplicité apparente se cachent des règles strictes, dont le non-respect a des conséquences directes sur l'argent du client.

2. Problématique

Comment concevoir une API REST qui garantit, en toutes circonstances, les invariants d'un produit d'épargne :

- Des montants toujours positifs,
- Un solde qui ne devient jamais négatif,
- Un blocage effectif des retraits jusqu'à l'échéance ou l'atteinte de l'objectif,
- Une étanchéité complète entre les données de chaque client ?

3. Objectifs du projet

3.1 Objectifs métier

- Fournir à LICELI Technologies un service backend fonctionnel de gestion de tirelires d'épargne, exposable à une future application cliente.
- Garantir la fiabilité et la sécurité des opérations financières, condition indispensable à la confiance des utilisateurs finaux.

3.2 Objectifs pédagogiques

- Modéliser un domaine métier et gérer l'évolution du schéma de données par les migrations.
- Exposer des ressources en REST avec Django REST Framework.
- Placer les règles métier au bon niveau, hors des vues.
- Mettre en œuvre l'authentification et le cloisonnement des données.

- Écrire des tests automatisés qui vérifient les refus autant que les cas nominaux.

4. Description fonctionnelle

L'application gère des tirelires. Une tirelire appartient à un client, porte un libellé, un montant objectif et une date d'échéance. Le client y enregistre des opérations de dépôt ou de retrait.

4.1 Les règles de gestion fondamentales

Règles applicables aux opérations (dépôt et retrait) :

- Le montant d'une opération est toujours strictement positif.
- Le solde d'une tirelire ne devient jamais négatif.
- Un retrait est refusé tant que l'objectif n'est pas atteint et que la date d'échéance n'est pas arrivée.
- Un client ne consulte et ne manipule que ses propres tirelires.

Règles applicables à la création d’une tirelire :

- Le montant objectif doit être strictement positif.
- La date d’échéance doit être postérieure à la date de création (dans le futur).

Ces règles doivent être appliquées par le serveur, indépendamment de ce que ferait une éventuelle interface, et refuser toute opération invalide par un message clair plutôt que par une erreur technique.

4.2 Fonctionnalités attendues

- Authentification du client
- Création d'une tirelire
- Consultation de la liste des tirelires d'un client
- Consultation du détail d'une tirelire
- Dépôt sur une tirelire
- Retrait sur une tirelire
- Consultation de l'historique des opérations d'une tirelire

5. Périmètre

5.1 Inclus

- La modélisation des données (Client, Tirelire, Opération)
- L'API de consultation et d'écriture (endpoints REST)
- L'application des règles de gestion (opérations et créations)
- L'authentification et le cloisonnement des données par client
- Les tests automatisés

5.2 Explicitement exclu

- L'intégration d'un moyen de paiement réel
- Toute interface graphique
- L'envoi de notifications

6. Contraintes

6.1 Contraintes techniques

- Langage et framework : Python 3.13, Django 5, Django REST Framework
- Base de données : SQLite en développement
- Gestion de version : Git

6.2 Contraintes de représentation des données

- Les sommes d'argent doivent être représentées avec un type décimal (Decimal), jamais avec un nombre à virgule flottante, afin d'éviter les erreurs d'arrondi inhérentes à la représentation binaire des flottants.

6.3 Contraintes de délai

- Durée totale : 2 mois, dont 4 semaines de développement structuré, le reste étant consacré à la consolidation, aux tests approfondis et à la préparation de la restitution finale.

6.4 Contraintes organisationnelles

- Point de suivi hebdomadaire avec le responsable de stage, consacré à la revue de code et à la levée des difficultés rencontrées.

7. Livrables attendus

- Un dépôt Git contenant le code source, avec un historique de commits lisible.
- Une API fonctionnelle, démarrable par une suite de commandes documentées.
- Un jeu de tests automatisés couvrant l'ensemble des règles métier (opérations et créations).
- Un fichier de prise en main décrivant l'installation, le démarrage et les points d'entrée de l'API.
- Les documents de conception associés au cycle de développement.

8. Critères d'évaluation

L'évaluation ne porte pas sur le volume de code produit, mais sur la solidité des règles de gestion. Cinq situations seront soumises à l'API en fin de projet. Chacune doit être refusée proprement, avec un message compréhensible et un code de retour HTTP approprié, sans erreur serveur :

- Enregistrer un dépôt d'un montant négatif ou nul
- Retirer un montant supérieur au solde disponible
- Retirer avant que l'objectif soit atteint et avant l'échéance
- Consulter ou modifier la tirelire d'un autre client
- Déposer sur une tirelire inexistante

Seront également appréciés la lisibilité du code, la pertinence du placement des règles métier, et la qualité des messages rendus à l'appelant.

9. Point d'attention techniques identifiées en cours de projet

- Représentation des montants : usage systématique du type Decimal (Django DecimalField), jamais float, pour toute donnée monétaire.
- Gestion du solde : solde stocké sur la tirelire, mis à jour dans la même transaction que l'opération associée, plutôt qu'un recalcul systématique à partir de l'historique.

Table des matières

Sommaire 2

Glossaire et liste des sigles 3

1. Contexte 4

2. Problématique 4

3. Objectifs du projet 4

3.1 Objectifs métier 4

3.2 Objectifs pédagogiques 4

4. Description fonctionnelle 5

4.1 Les règles de gestion fondamentales 5

4.2 Fonctionnalités attendues 5

5. Périmètre 6

5.1 Inclus 6

5.2 Explicitement exclu 6

6. Contraintes 6

6.1 Contraintes techniques 6

6.2 Contraintes de représentation des données 6

6.3 Contraintes de délai 6

6.4 Contraintes organisationnelles 6

7. Livrables attendus 7

8. Critères d'évaluation 7

9. Point d'attention techniques identifiées en cours de projet 7

Table des matières 8